

WebXR + Video

Prepared for syd<video>

by Kevin Staunton-Lambert
@keвлеyski

pyrmontbrewery.com.au



WA



WebXR



Firstly, what's this all about?

In a nutshell, how to watch immersive 3D motion video (e.g. 360°) using a Virtual (VR) or overlaying an Augmented Reality device



XR concepts

eXtended Reality - Virtual, Augmented Mixed

3D/immersive video media - For ease of packaging it up as a regular video - that is, ease of video compression and transport
streaming media - 360° video typically appears in its raw form as if looking at a 2D paper world map vs holding a physical globe

XR concepts - Rectilinear 360° video projection



XR concepts - Rectilinear 360° video projection



XR concepts

When
rectilinear
is wrapped
around a
ball, things
are back in
3D again



XR concepts - immersive / surrounding

Immersive - being inside that ball looking out

3D stereoscopic doubles things up for left/right eyes (side-by-side or top/bottom)

VR180 is for when what's behind you really doesn't add to the experience (mostly better quality, than saving bandwidth)



OK, what else is immersive? Spatial Audio

Audio is essential to track with the video to avoid any dissociation (can lead to nausea)

Ambisonics binaural/surround
point space "virtual speakers"
Sound is re-modulated as head
orientation changes - Doppler
amp, phase, freq manipulation



OK, what else is immersive? Haptic Peripherals

Other immersive peripherals like haptic bodysuits that might be used for both input (pose) or output to inflict a physical punch or "other things" some, pretty intimate!

Other senses considered like applying warmth/smells perhaps for sensory training and mindfulness applications



Augmented Reality video?

XR devices include room walls as boundaries but also fixed objects like doors, windows
couches can be included in a XR space model

AR training applications

AR live targeted video ad replacement maybe!



Great OK, What's WebXR?

W3C standard for eXtended Reality devices

Does **not** render VR/AR gfx or video itself, i.e. no shaders/meshes/materials etc, but... does provide a frame rendering context via WebGL

Immersive Virtual Reality session control

Immersive Augmented Reality sessions



What WebXR standardises across devices

Head pose (VR headset/AR glasses orientation)

Hand pose (naked finger/wrist positions, gesture)

Controller pose (orientation, gamepad buttons, triggers, squeeze+touch, thumbstick axes/touch)

Space control/tracking (3/6DoF travelling state)

Things like distance units, eye tracking, haptics

WebXR JS API (Firefox, Chrome, Meta, HTC, Magic Leap, Hololens, Apple?...

navigator.xr (handle to XRSystem object)

XRSession (async Promise when/if immersive can be currently supported by the client)

XRFrame (animation/object state machine)

XRInputSource (head/controllers/hands/gaze)

WebXR JS API ... etc, API applies to all reality devices)

➤ **XRSpace** (6DoF spatial tracking, teleporting)

Bounded reference space vs local/floor ref

XRLayer (abstraction to rendering frame buffers) of which **XRWebGLLayer** is a binding between a WebGL/canvas context and XRFrame which can be used to get pose which can be used to manipulate the camera

XRSession - enter and exit immersive reality

```
window.navigator.xr.isSessionSupported('immersive-vr').then((supportsVR) =>  
if (supportsVR) xrSession = window.navigator.xr.requestSession ... {
```

Start/stops VR/AR session taking in options like, use hand tracking, boundary/hit detection that applies to this reality session

All VR/AR apps have 'transient activation' e.g. "ENTER VR" or  which the user "presses" by aiming and shooting with the controller trigger

XRInputSource - Hand Controllers

Control ray emits from trigger, JS event listeners for 'select', 'squeeze' and axes array



Some controllers expose some less obvious sensors via WebXR (aids user anticipation):

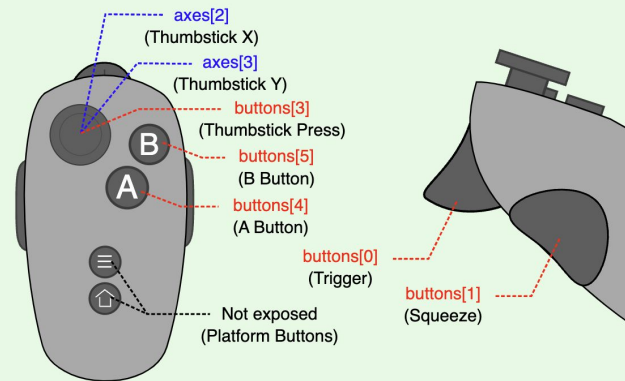
Touch/hover over all buttons

Stress plate (near buttons)

Touch tip of thumbstick

Very fine movements (Apple Pencil like)

Example complex 'xr-standard' controller



NOTE: For this device `axes[0]`, `axes[1]`, and `buttons[2]` are placeholders, since they are reserved for Touchpad inputs

XRInputSource - Gaze (head/eye only input)

Dalek mode, control ray emits from forehead

Can be used to support foveated-rendering, optimizing where the user is looking, pre-cached assets what their next likely action will likely be and render that



XRHand - input naked hand-tracking

WebXR supports 24 hand and wrist datapoints, ray emits from thumb, pinch to tap/click

Gloves are NOT necessary!



```
const wristJoint = inputSource.hand.get("wrist");  
const indexFingerTipJoint = inputSource.hand.get("index-finger-tip");  
  
for (const [joint, jointSpace] of inputSource.hand) { ...  
// see https://www.w3.org/TR/webxr-hand-input-1/
```



XRSpace - 6DoF Spatial tracking

Head/hands/controllers XRPose per object relative to XRSpace (as 4x4 matrices)

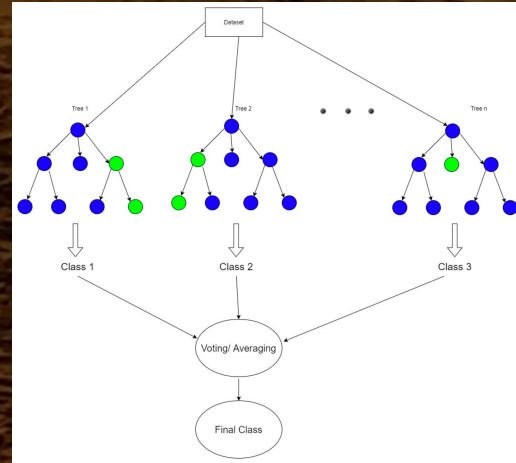
Teleporting (try Apple Maps around Pyrmont)

```
xrsession.requestReferenceSpace('bounded-floor').then(xrSpace) => space.boundsGeometry[i];  
  
// handedness and axes  
xrsession.inputSources.forEach( inputSource => {  
  const gp = inputSource.gamepad;  
  if ( inputSource.handedness == 'right'){  
    // update right hand, this vec3.set( gp.axes[offset], -gp.axes[offset + 1], 0 );  
  } else if ( inputSource.handedness == 'left'){  
    // update left hand, this vec3.set( gp.axes[offset], -gp.axes[offset + 1], 0 );  
  });
```

XRFrame - Holds object state to be rendered

Frame obtained from the XR animation loop represents the state of all tracked objects for any given time

```
function animationLoop(time, xrFrame) {  
    const viewer = xrFrame.getViewerPose(xrSpace);  
  
    webgl.render(scene, camera);  
    xrSession.requestAnimationFrame(animationLoop);  
}
```



OK, what about WebXR with motion video?

Web browsers that support **<video>** and **<canvas>** tags like Chromium, Edge, Safari, Firefox, Chrome can render

If you have a player app, e.g. Shaka, hls.js, dash.js, Theo Bitmovin you can get a handle to the **<video>** element

```
const video = document.querySelector("video");
```

Then create a **<canvas>** element and push **<video>** to it and manipulate **<canvas>** based on WebXR orientation...

HTML `<video>` → `<canvas>` bit blit

Basic MSE copy to canvas...

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<canvas id="canvas" width="640" height="480">
```

```
<video id="video" controls>
```

```
<source src="reckless_in_pymont.mp4" type="video/mp4">
```

```
</video>
```

`<canvas id="canvas">`

`<video id="video">`

HTML `<video>` → `<canvas>` bit blit (CanvasRenderingContext2D)

```
<script>
```

```
const canvas = document.getElementById('canvas');
```

```
const video = document.getElementById('video');
```

```
const canvas_context = canvas.getContext('2d');
```

```
video.addEventListener('play', function () {
```

```
    const self = this;
```

```
    function blit_video_to_canvas () {
```

```
        canvas_context.drawImage(self, 0, 0);
```

```
        video.requestVideoFrameCallback(blit_video_to_canvas);
```

```
    }
```

```
    blit_video_to_canvas();
```

```
});
```

```
</script>
```

HTML `<video>` → `<canvas>` bit blit MSE video

```
<!DOCTYPE html>
<html>
  <body>
    <canvas id="canvas" width="640" height="480"></canvas>
    <video id="video" controls>
      <source src="reckless_in_pymont.mp4" type="video/mp4">
    </video>
  </body>
</html>

<script>
  const canvas = document.getElementById('canvas');
  const video = document.getElementById('video');
  const canvas_context = canvas.getContext('2d');
  video.addEventListener('play', function () {
    const self = this;

    function blit_video_to_canvas () {
      canvas_context.drawImage(self, 0, 0);
      video.requestVideoFrameCallback(blit_video_to_canvas);
    }
    blit_video_to_canvas();
  });
</script>
</body>
</html>
```



HTML `<video>` → `<canvas>` rendering video in 3D?

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<canvas id="canvas"
```

```
style="transform:
```

```
perspective (456px)
```

```
rotateX (245deg)
```

```
filter:
```

```
); "
```

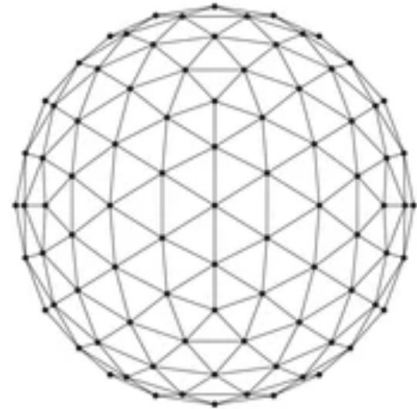
**To the pub!
we're done??
too easy!...**



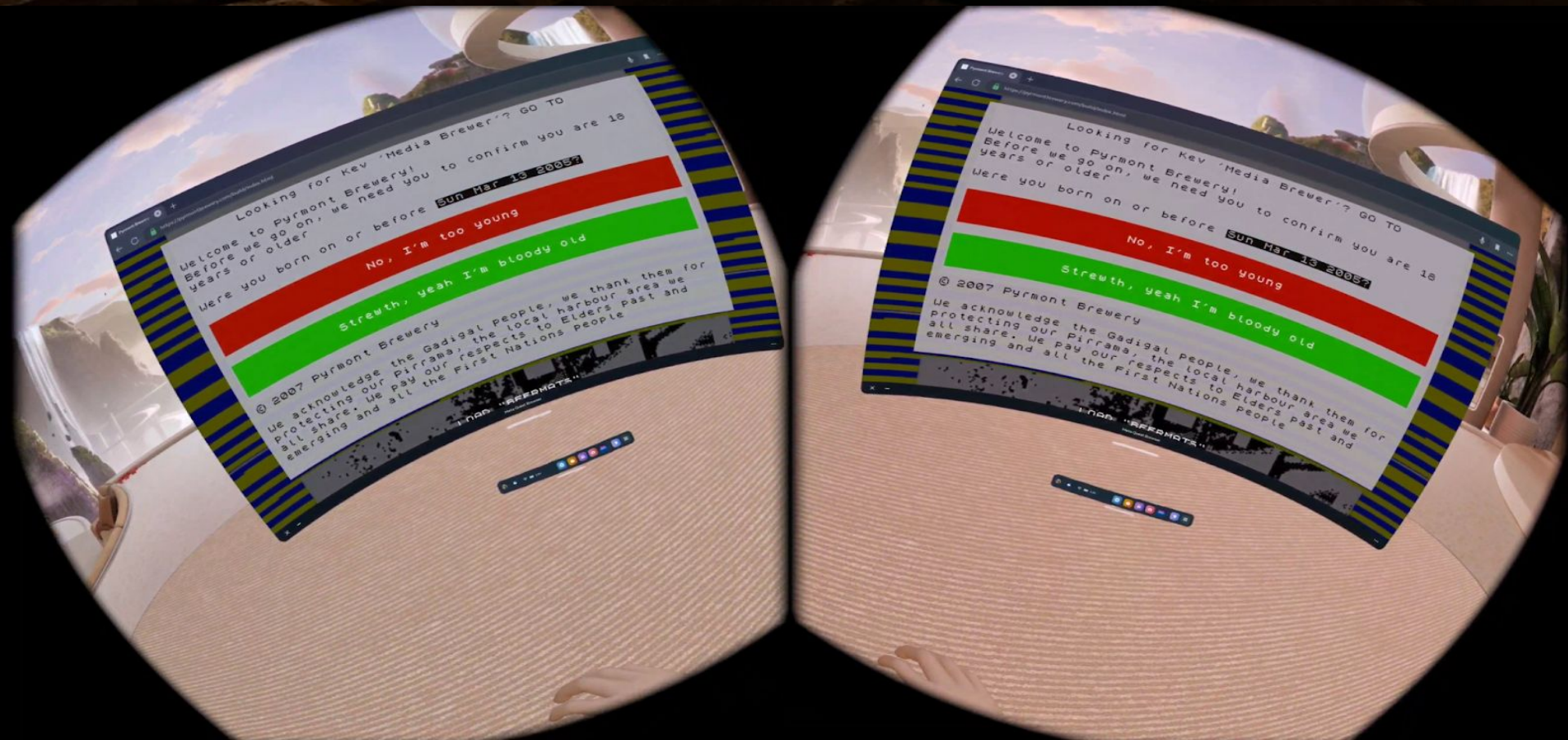
Yeah, nah! video part *works*, but that's **not** 3D :-)

Rendering a 2D <canvas> on a headset has no **perspective** (depth) or **parallax** (stereo eyes)

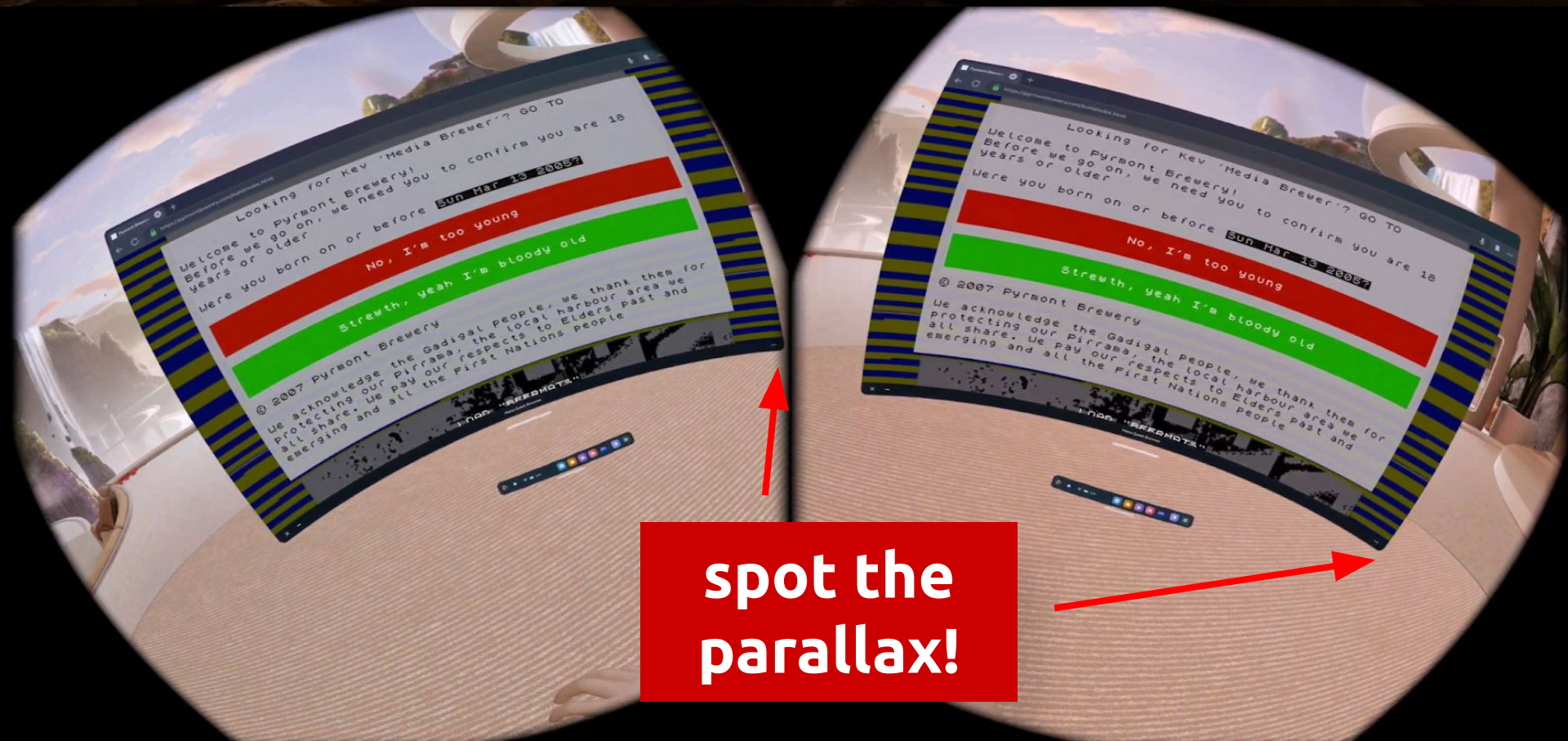
Immersive 360° video needs a ball to project onto - in graphic primitives a ball is many triangles (icosahedron or UV sphere rings)



Actual headset frame buffer (Meta Quest Pro)



Actual headset frame buffer (Meta Quest Pro)



We need a virtual planetarium!

Rather than bit blit the `<video>` tag *directly*,
we indirect copy video as a 2D texture, *then*
have **WebGL2** project this in the `<canvas>` tag

The most *efficient* way to
do this is **WebAssembly**
The *tightest/secure* Wasm
code is generated by **Rust!**



Rendering to WebGL with Rust / WebAssembly

This won't hurt! - easier than you might think

```
# Get Rust and wasm-pack
```

```
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh -s -- -y
```

```
curl https://rustwasm.github.io/wasm-pack/installer/init.sh -sSf | sh
```

Create a dynamic Rust library 'webxr_webgl'

```
cargo new --lib webxr_webgl
```

```
[lib]
```

```
crate-type = ["cdylib", "rlib"]
```

(cdylib = extern C dynamic linked library - for wasm)



Rendering to WebGL with Rust / WebAssembly

Add in the **wasm-bindgen** Rust crate

```
[dependencies]
```

```
wasm-bindgen = "0.2.85"
```

Run **wasm-pack** (cargo build with wasm target)

```
wasm-pack build --target web
```

(generates ./pkg with wasm binary + JS)



WA

Rendering to WebGL with Rust / WebAssembly

Add to HTML/JS to import

pkg/webxr_webgl.js

```
<script type="module">
```

```
import init from "webxr_webgl.js";
```

```
init().then((wasm_exports) => {
```

```
  console.log("webxr_webgl.wasm loaded")
```

```
});
```

```
</script>
```



WA

Rendering to WebGL with Rust / WebAssembly

`wasm_exports` = our pub externs as JS methods + props

Rust functions exported using `wasm_bindgen` macro

```
#[wasm_bindgen]
```

```
pub fn foo(some_string_from_javascript &str) {
```

Rust code can eval JS functions

```
#[wasm_bindgen]
```

```
extern { pub fn confirm(message: &str); }
```

Async JS Promise as Rust Futures too

```
wasm-bindgen-futures js_sys::Promise
```



WA

Rendering to WebGL with Rust / WebAssembly

Ok, Rust/WASM/JS bindings sorted, next things...

Share **<canvas>**, add video sphere vertices + shader

Basic sin/cos trig but using old school ZX Spectrum

look-up table trick of only needing

quarter of the calculations as it's a

symmetrical shape we can mirror



WA

Rendering to WebGL with Rust / WebAssembly

Add **web-sys** Rust crate to access JS DOM + WebGL

```
[dependencies.web-sys]
version = "0.3.4"
features = [
    'Window',
    'Document',
    'HtmlCanvasElement',
    'WebGl2RenderingContext',
    'WebGlBuffer',
    'WebGlVertexArrayObject',
    'WebGlProgram',
    'WebGlShader',
]
```



WA

WebXR API - setup WebGL2 context + renderer

```
const xr_gl = canvas.getContext('webgl2'); // JS
document.canvas.getContext

navigator.xr.requestSession('immersive-vr', {requiredFeatures: ['hit-test'], optionalFeatures:
['local-floor']})
.then(onSessionStarted);

function onSessionStarted(xrSession) {
  setupWebGLLayer()
    .then(() => {
      animate();
    })
}

function setupWebGLLayer() {
  return xr_gl.makeXRCompatible().then(() => {
    xrSession.updateRenderState( {baseLayer: new XRWebGLLayer(xrSession, gl) });
  });
}

function animate() {
  renderer.setAnimationLoop(render);
}
```



WebXR

Rendering to WebGL with Rust / WebAssembly

Access that *same* `xr_g1` via `document.<canvas>` from Rust

```
#[wasm_bindgen(start)]
fn start() -> Result<(), JsValue> {
    let document = web_sys::window().unwrap().document().unwrap();
    let canvas = document.get_element_by_id("canvas").unwrap();
    let canvas: web_sys::HtmlCanvasElement =
        canvas.dyn_into:::<web_sys::HtmlCanvasElement>()?;

    // Rust equiv document.canvas.getContext
    let xr_g1 = canvas.get_context("webgl2")?
        .unwrap()
        .dyn_into:::<WebGl2RenderingContext>()?;
```



WA

WebGL2 GLSL shader for video bit blit

WebGL2 = GLES (OpenGL ES) 3.0

OpenGL maintains a state of buffers and shaders and renders using hardware GPU

Vertex shader (where to render, clip regions)

Fragment shader (how to render pixels)



WebGL2 GLSL shader for video bit blit

Video GLES 3.0 vertex shader (basic vec4 copy)

```
let video_vertex_shader = compile_shader(  
  &xr_gl,  
  WebGL2RenderingContext: VERTEX_SHADER,  
  r##"#version 300 es  
  in vec4 position;  
  void main() {  
    gl_Position = position;  
  }  
  "##,  
  )?;
```



WebGL2 GLSL shader for video bit blit

Video GLES 3.0 Fragment shader (coloured video)

```
let video_fragment_shader = compile_shader(  
    &xr_gl,  
    WebGL2RenderingContext: FRAGMENT_SHADER,  
    #version 300 es  
    varying highp vec2 vTextureCoord;  
    varying highp vec3 vLighting;  
    uniform sampler2D uSampler;  
    void main(void) {  
        highp vec4 texelColor = texture2D(uSampler, vTextureCoord);  
        gl_FragColor = vec4(texelColor.rgb * vLighting, texelColor.a)  
    }  
`;  
// uSampler - a uniform global that holds the video texture  
// vTextureCoord - texture coord for vertex (passed into shader)
```



WebGL2 GLSL shader for video bit blit

Compile/attach video shader as GLES program

```
let gles_program = xr_gl.create_program().ok_or_else(|| String::from("Doh!"))?;  
xr_gl.attach_shader(&program, video_vertex_shader);  
xr_gl.attach_shader(&program, video_fragment_shader);  
xr_gl.link_program(&program);  
  
xr_gl.use_program(Some(&gles_program));
```

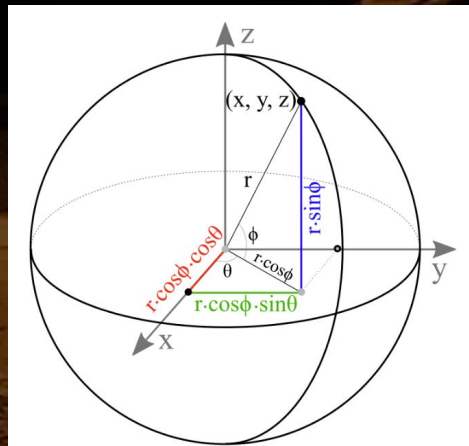
(this set the GLES state up, whatever draws next apply this shader program)



Rendering to WebGL with Rust / WebAssembly

Planetarium indexed vertices buffer obj

```
let planetarium_vtxarr_obj = xr_gl.create_vertex_array();  
// generate vertices, normals and uvs  
for vertex in ring_of_sphere_iter {  
    // vertex  
  
    vertex.x = -radius * cos( phiStart + u * phiLength ) * sin( thetaStart + v * thetaLength );  
    vertex.y = radius * cos( thetaStart + v * thetaLength );  
    vertex.z = radius * sin( phiStart + u * phiLength ) * sin( thetaStart + v * thetaLength );  
  
    // indices  
  
    const vbo1 = grid[ iy ][ ix + 1 ];    const vbo2 = grid[ iy ][ ix ];  
    const vbo3 = grid[ iy + 1 ][ ix ];    const vbo4 = grid[ iy + 1 ][ ix + 1 ];  
  
    if ( iy !== 0 || thetaStart > 0 ) {  
        xr_gl.bufferData(xr_gl.ARRAY_BUFFER, new Float32Array([vbo1, vbo2, vbo4]), gl)  
    }  
    if ( iy !== heightSegments - 1 || thetaEnd < PI ) {  
        xr_gl.bufferData(xr_gl.ARRAY_BUFFER, new Float32Array([vbo2, vbo3, vbo4]), gl)
```



Rendering to WebGL with Rust / WebAssembly

Bind planetarium array buffer to **<canvas>** GL context

```
xr_gl.bind_buffer(WebGl2RenderingContext::ARRAY_BUFFER, null);  
xr_gl.bind_vertex_array(Some(&planetarium_vtxarr_obj));  
// Tell WebGL how to pull out the VBO from the buffer into the vertexPosition attr  
xr_gl.enable_vertex_attrib_array(position_attribute_location as u32);
```

Draw the planetarium to **<canvas>**

```
xr_gl.clear_color(0.0, 0.0, 0.0, 1.0);  
xr_gl.clear(WebGl2RenderingContext::COLOR_BUFFER_BIT);  
xr_gl.draw_arrays(WebGl2RenderingContext::TRIANGLES, 0, ver
```



Rendering to WebGL with Rust / WebAssembly

Why we're doing this... LLVM compiled code means **optimised** code (typical performance is more than 4x faster than any interpreted JavaScript will ever be)

Common op-codes now shared with **hardware architecture** (improved h/w acceleration options, op-codes limited to sandbox) and access accelerated GPU and libraries and DRM (trusted) decoder pipelines

Teeny tiny code, super fast to download, small storage/RAM runtime footprint *...but as you can see*

... bendy XR video gets complicated quickly!

```
let program = xr_gl  
    .create_program ()  
    .ok_or_else (|| String::from("Unable to create shader object" ))?;
```

```
xr_gl.attach_shader (&program, vert_shader);  
xr_gl.attach_shader (&program, frag_shader);  
xr_gl.link_program (&program);
```

```
if xr_gl  
    .get_program_parameter  
    .as_bool ()  
    .unwrap
```

```
xr  
xr
```

```
xr_g  
xr_g
```

```
xr_gl.  
WebC  
0,  
(vert  
);  
Ok ( ) )
```

Rust + WebAssembly is by far the
most **efficient** way... but is there an
easier way to render 3D video?
yes! there is!...



WA



three.js and AFrame.io to the rescue!

three.js - JS framework that wraps WebGL
It also has a library of controller meshes
(XRControllerModelFactory) and locomotion
vs teleporting/travel arcs and spatial audio

A-Frame - helper framework that wraps
three.js with WebXR! Easy client side face
landmark tracking

```
<a-scene webxr="... systems.webxr."
```


video.js VR plug-in (WebXR update)

StreamShark sponsored free open source!

Add WebXR session control/animation loop

Add immersive video controls to Video.JS

Immersive Video Projections:

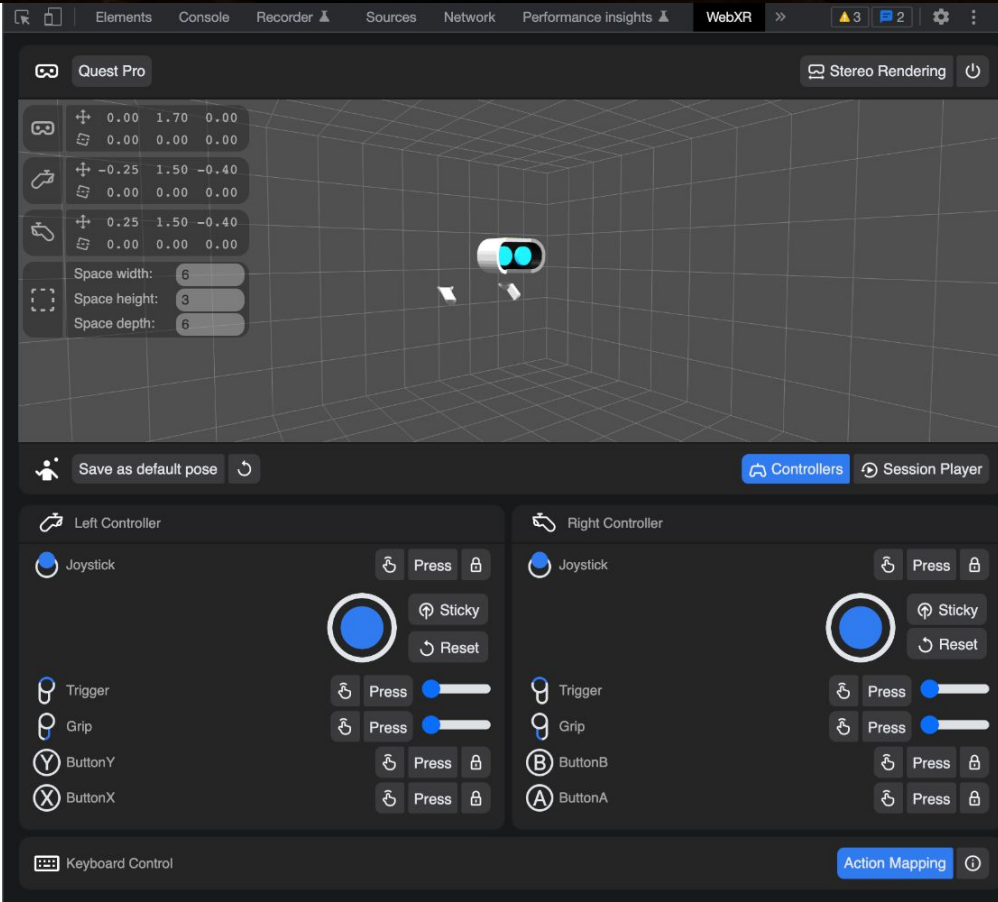
Spherical Rectilinear VR180° T/B, VR180° L/R

Cubemap (aka full motion video skyboxes)

video.js VR plug-in (WebXR update)



- [Run unit tests in browser](#)
- [180 video example \(left/right\)](#)
- [180 video example \(top/bottom\)](#)
- [180 video example \(monoscopic input\)](#)
- [360 Cube Video example](#)
- [360 Video example](#)
- [Another 360 Video example](#)
- [360 TB Video example](#)
- [EAC video example](#)
- [EAC LR video example](#)
- ["Fluid" video size example](#)
- [Spatial audio example](#)
- [360 with increased sphere detail example](#)



What's coming to WebXR - WebGPU!

WebGPU (navigator.gpu) has better cross compatibility with modern GPUs, Metal/Vulkan

Faster GPU computations/operations (less CPU)

Simplifies GPGPU (non-gfx) parallel compute shader modules

(faster computer vision, codecs etc)



WebGPU

What's coming to WebXR - wgpu and dawn

▶ Cross GPU shader language **WGSL** (Metal inf)

Big advances in **video AI**, various WebGPU libraries are already supporting CNNs and ONNX models (see wonnx Rust / WebGPU) and Apache tvm for examples

WebXR support still early days for XRFrame / XRGPUSubImage in dev



What's new? Apple Vision WebXR

WebGPU and WGSL were heavily influenced by Metal. WebKit has been adding WebXR support - we should expect Apple VR headset or AR glasses to have WebXR via Embedded Safari



Metal

WebXR Augmented Reality Module

WebXR Device API

WebXR Gamepads Module

WebXR Hand Input Module

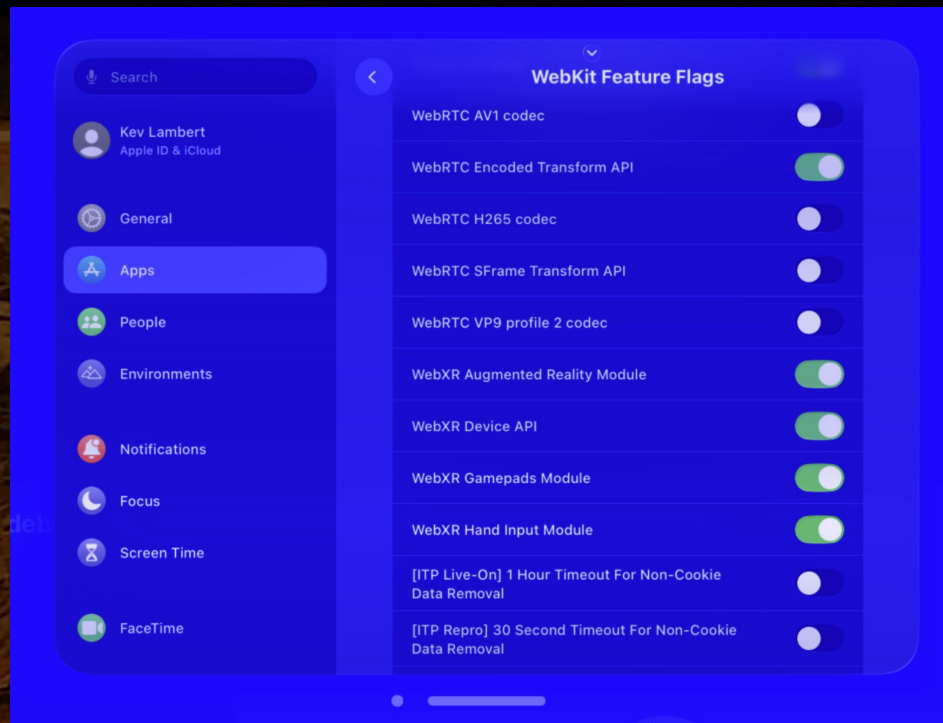


WebGPU

What's new? Apple Vision WebXR

WebXR is **not** yet enabled by default so you have to turn it on...

Do NOT setup a room environment stick with the default blue, I hit a lot of crashes and reboots with that



Also, XR over 5G Edge Servers! CloudXR render

OpenXR will enable AR applications to run portably on edge server architectures

Significant industry effort in developing this use case

Wireless mobile device with display and sensors



Runtime using 5G implemented across device and server and accessed through OpenXR API

5G Low latency Sensor Data

Sensor handling

OpenXR™

Display composition

5G Generated Augmentations & Scenes

Vulkan.



MEC (Multi-access Edge Computing) Server

1. Processes sensor data, can include machine learning for environmental lighting, occlusion, scene semantics, object reconstruction and UI
2. Generates imagery from 3D models, can include stereo, foveal rendering, ray-tracing, optics pre-distortion, varifocal processing



XR over 5G Edge Servers! (shared cloud render)

Distributed edge computing reduces latency

Higher complexity rendering, shared arenas etc

Great news for eSports gaming applications!

Live media sports betting / bidding auctions

Low latency VR video in metaverse worlds

Live in-game ad targeting/rendering at scale

References further reading:

W3C Specs: <https://www.w3.org/TR/webxr> <https://www.w3.org/TR/WGSL/>

Rust/WebAssembly wasm-bindgen/pack - <https://github.com/keveleski/wasm->*

WebGL wasm: <https://rustwasm.github.io/docs/wasm-bindgen/examples/webgl.html>

WebGPU: <https://github.com/immersive-web/WebXR-WebGPU-Binding/blob/main/explainer.md>

Chrome WebXR emulator:

<https://chrome.google.com/webstore/detail/immersive-web-emulator/cgffilbpcibhmcfbgggfhfolhkfbhmik>

Mozilla Firefox WebXR API: <https://addons.mozilla.org/en-US/firefox/addon/webxr-api-emulator>

mdn: <https://developer.mozilla.org/en-US/docs/Web/API/XRHand>

Keys Video.js VR: <https://github.com/videojs/videojs-vr/pull/273>

Thanks!

